

Desenvolvimento para Dispositivos Móveis

Consumindo dados de uma API JSON com JWT

Profº: Joseph Donald

Contatos:

☎ (83) 98228-8607

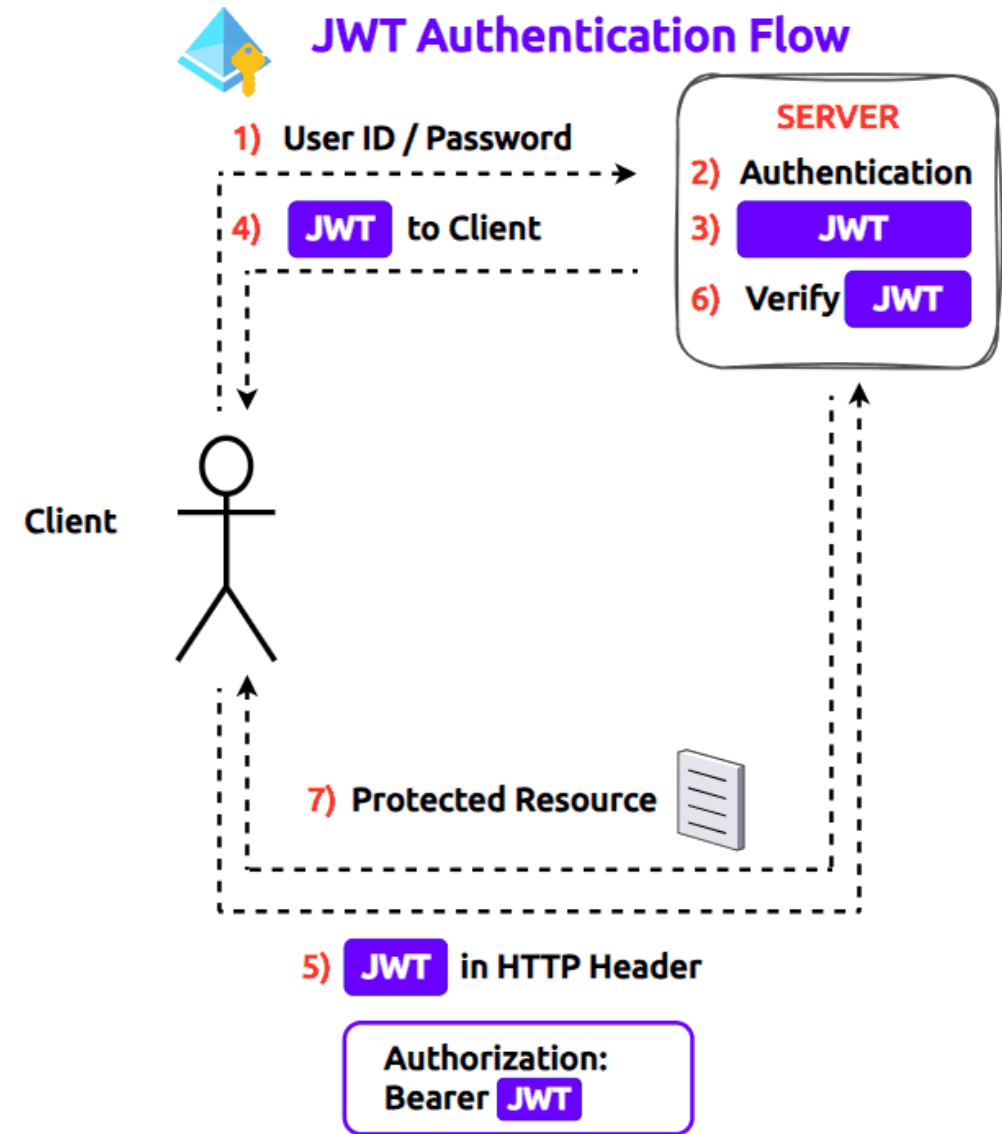
📷 @josephdonald

✉ 030106382@prof.uninassau.edu.br

“Se você tem uma maçã e eu tenho outra; e nós trocamos as maçãs, então cada um terá sua maçã. Mas se você tem uma ideia e eu tenho outra, e nós as trocamos; então cada um terá duas ideias.”

George Bernard Shaw

- O JWT (JSON Web Token) no Flutter funciona como um “cartão de identificação digital”.
- Ele é gerado pelo seu servidor backend após o login do usuário e usado pelo seu app Flutter em requisições subsequentes para autenticar e autorizar o acesso a rotas protegidas.



- O Flutter não deve manter o token na memória. Ele deve ser **armazenado localmente** e, no caso do mobile, de forma criptografada para que não seja perdido quando o app for fechado ou roubado por outro app do celular.
- Para armazenar o token utilizando a versão Flutter Mobile utilizasse o pacote **Flutter Secure Storage**
- Para armazenar o token utilizando a versão Flutter Web utilizasse o pacote **SharedPreferences**



Server

JSON Web Token



Client

Rahul Golwalkar

- Para nosso exemplo prático, como utilizamos o Flutter Web, utilizaremos o **Shared Preferences** para guardar o token.
- **Lembrete:**
 - ✓ **Comando para instalação do Shared Preferences**

```
flutter pub add shared_preferences
```
- Uma vez que o Shared Preferences se encontra instalado no projeto, ele vai atuar na gravação e leitura do token conforme for necessário.



Exemplos: LOGIN

- Ao fazer o login o token é enviado pelo backend para o app Flutter. O app vai **receber o token e salvar** em um Shared Preferences.

```
38     if (response.statusCode == 200) {  
39         // Login bem-sucedido  
40         final responseDadosUsuario = jsonDecode(response.body)[0];  
41  
42         String token = responseDadosUsuario["token"];  
43         // Guardar o token em SharedPreferences  
44         SharedPreferences prefs = await SharedPreferences.getInstance();  
45         await prefs.setString('token', token);  
    }
```

CRUD

- Uma vez estando logado, quando o usuário **precisar executar alguma tarefa protegida pelo JWT**, o app Flutter deverá fazer a **leitura do Token** e enviar junto com a requisição correspondente a essa tarefa.

```
23     // Pegar token JWT do SharedPreferences  
24     Future<String?> getToken() async {  
25         SharedPreferences prefs = await SharedPreferences.getInstance();  
26         return prefs.getString('token');  
27     }
```

```
61     //FUNÇÃO CADASTRA FUNCIONARIO  
62     Future<void> cadastraFuncionario(Map<String, dynamic> mapFuncionario) async {  
63         String? token = await getToken();  
64  
65         var response = await http.post(  
66             Uri.parse('http://$ip_servidor:5000/cadastraFuncionario'),  
67             headers: {  
68                 'Accept': 'application/json',  
69                 'content-type': 'application/json',  
70                 'Authorization':  
71                     'Bearer $token', // Enviar o token JWT no cabeçalho Authorization  
72             },  
73             body: jsonEncode(mapFuncionario),  
74             encoding: Encoding.getByName('utf-8'),  
75         );  
    }
```

Exemplos:

LOGOUT

- Quando o usuário fizer o logout o app deverá **remover o token do Shared Preferences**.

```
155 // Função de Logout
156 Future<void> logout() async {
157   setState(() {
158     listaFuncionarios = [];
159     dadosUsuario = null;
160   });
161
162   SharedPreferences prefs = await SharedPreferences.getInstance();
163   await prefs.remove('token'); // Remove o token JWT do SharedPreferences
```





Na função autenticaUsuario()

```
27 Future<void> autenticaUsuario(Map<String, dynamic> mapLogin) async {
28   print("MAPA LOGIN: " + mapLogin.toString());
29   var response = await http.post(
30     Uri.parse('http://$ip_servidor:5000/loginUsuario'),
31     headers: {
32       'Accept': 'application/json',
33       'content-type': 'application/json',
34     },
35     body: jsonEncode(mapLogin),
36     encoding: Encoding.getByName('utf-8'),
37   );
38
39   if (response.statusCode == 200) {
40     // Login bem-sucedido
41     final responseDadosUsuario = jsonDecode(response.body)[0];
42
43     String token = responseDadosUsuario["token"];
44     // Guardar o token em SharedPreferences
45     SharedPreferences prefs = await SharedPreferences.getInstance();
46     await prefs.setString('token', token);
47
48     ScaffoldMessenger.of(context).showSnackBar(
49       SnackBar(content: Text(responseDadosUsuario["mensagem"].toString())),
50     );
51     Navigator.pushReplacement(
52       context,
53       MaterialPageRoute(
54         builder: (context) => const HomeView(),
55         settings: RouteSettings(arguments: responseDadosUsuario),
56       ), // MaterialPageRoute
57     );
58     print('Usuário cadastrado com sucesso!');
59   } else {
60     // Erro no cadastro
61     Map<String, dynamic> responseDadosUsuario = jsonDecode(response.body);
62     ScaffoldMessenger.of(context).showSnackBar(
63       SnackBar(content: Text(responseDadosUsuario["mensagem"].toString())),
64     );
65     print('Erro ao cadastrar usuário: ${response.body}');
66   }
67 }
```

Implementação da função `getToken()`

```
22  
23 // Pegar token JWT do SharedPreferences  
24 Future<String?> getToken() async {  
25   SharedPreferences prefs = await SharedPreferences.getInstance();  
26   return prefs.getString('token');  
27 }
```

Na função `carregaDados()`

```
29 //FUNÇÃO CARREGA FUNCIONARIOS NA LISTA  
30 Future<void> carregaDados() async {  
31   String? token = await getToken();  
32  
33   if (token == null || token.isEmpty) {  
34     return;  
35   }  
36  
37   var dadosRecebidos = await http.get(  
38     Uri.parse('http://$ip_servidor:5000/lerFuncionario'),  
39     headers: {  
40       'Accept': 'application/json',  
41       'content-type': 'application/json',  
42       'Authorization':  
43         'Bearer $token', // Enviar o token JWT no cabeçalho Authorization  
44     },  
45   );  
46  
47   if (dadosRecebidos.statusCode == 200) {  
48     print("DADOS RECEBIDOS: " + dadosRecebidos.body);  
49  
50     setState(() {  
51       listaFuncionarios = jsonDecode(dadosRecebidos.body)[0];  
52     });  
53   } else {  
54     ScaffoldMessenger.of(  
55       context,  
56     ).showSnackBar(SnackBar(content: Text("Erro ao carregar dados!!")));  
57     print("ERRO AO CARREGAR DADOS: " + dadosRecebidos.body);  
58   }  
59 }
```


Na função `cadastraFuncionario( )`

```
60
61 //FUNÇÃO CADASTRA FUNCIONÁRIO
62 Future<void> cadastraFuncionario(Map<String, dynamic> mapFuncionario) async {
63   String? token = await getToken();
64
65   var response = await http.post(
66     Uri.parse('http://$ip_servidor:5000/cadastraFuncionario'),
67     headers: {
68       'Accept': 'application/json',
69       'content-type': 'application/json',
70       'Authorization':
71         'Bearer $token', // Enviar o token JWT no cabeçalho Authorization
72     },
73     body: jsonEncode(mapFuncionario),
74     encoding: Encoding.getByName('utf-8'),
75   );
76
77   if (response.statusCode == 200) {
78     ScaffoldMessenger.of(
79       context,
80     ).showSnackBar(SnackBar(content: Text("Cadastrado com sucesso!!")));
81     setState(() {
82       carregaDados();
83     });
84     Navigator.pop(context);
85   } else {
86     // Erro no cadastro
87     ScaffoldMessenger.of(
88       context,
89     ).showSnackBar(SnackBar(content: Text("Erro ao cadastrar usuário!!")));
90   }
91 }
```

Na função `atualizaFuncionario( )`

```
93 //FUNÇÃO ATUALIZA FUNCIONÁRIO
94 Future<void> atualizaFuncionario(Map<String, dynamic> mapFuncionario) async {
95   String? token = await getToken();
96
97   var response = await http.put(
98     Uri.parse('http://$ip_servidor:5000/atualizaFuncionario'),
99     headers: {
100       'Accept': 'application/json',
101       'content-type': 'application/json',
102       'Authorization':
103         'Bearer $token', // Enviar o token JWT no cabeçalho Authorization
104     },
105     body: jsonEncode(mapFuncionario),
106     encoding: Encoding.getByName('utf-8'),
107   );
108
109   if (response.statusCode == 200) {
110     ScaffoldMessenger.of(
111       context,
112     ).showSnackBar(SnackBar(content: Text("Atualizado com sucesso!!")));
113     setState(() {
114       carregaDados();
115     });
116   } else {
117     // Erro no cadastro
118     ScaffoldMessenger.of(
119       context,
120     ).showSnackBar(SnackBar(content: Text("Erro ao atualizar usuário!!")));
121   }
122 }
```

Na função `excluirFuncionario( )`

```
124 //FUNÇÃO EXCLUIR FUNCIONÁRIO
125 Future<void> excluirFuncionario(Map<String, dynamic> mapFuncionario) async {
126   String? token = await getToken();
127
128   var response = await http.delete(
129     Uri.parse('http://$ip_servidor:5000/deletaFuncionario'),
130     headers: {
131       'Accept': 'application/json',
132       'content-type': 'application/json',
133       'Authorization':
134         'Bearer $token', // Enviar o token JWT no cabeçalho Authorization
135     },
136     body: jsonEncode(mapFuncionario),
137     encoding: Encoding.getByName('utf-8'),
138   );
139
140   if (response.statusCode == 200) {
141     ScaffoldMessenger.of(
142       context,
143     ).showSnackBar(SnackBar(content: Text("Excluído com sucesso!!")));
144     setState(() {
145       carregaDados();
146     });
147   } else {
148     ScaffoldMessenger.of(
149       context,
150     ).showSnackBar(SnackBar(content: Text("Erro ao excluir usuário!!")));
151   }
152   Navigator.pop(context);
153 }
```



Na função `logout()`

```
155 // Função de Logout
156 Future<void> logout() async {
157   setState(() {
158     listaFuncionarios = [];
159     dadosUsuario = null;
160   });
161
162   SharedPreferences prefs = await SharedPreferences.getInstance();
163   await prefs.remove('token'); // Remove o token JWT do SharedPreferences
164
165   ScaffoldMessenger.of(
166     context,
167   ).showSnackBar(SnackBar(content: Text("Logout realizado com sucesso!!")));
168
169   Navigator.pushReplacement(
170     context,
171     MaterialPageRoute(builder: (context) => const LoginView()),
172   );
173 }
```